

Networking & Security Hardening

Category: **Cloud & Network Security** | Platform Standard: **Linux 6+ / Modern Cloud SRE**

1. Domain Architectural Overview

This official module provides comprehensive engineering depth and runnable syntax for **Networking & Security Hardening**. It is strictly scoped to this domain to prevent knowledge cross-contamination. Engineers mastering this domain acquire hands-on proficiency from foundational setup to production-grade incident triage and resilience.

2. Core Concepts & Production Principles

- **TCP 3-Way Handshake & Socket Lifecycle:** Connection establishment via SYN, SYN-ACK, ACK. Teardown via FIN/ACK sequences. States include ESTABLISHED, TIME_WAIT (waiting for delayed packets), and CLOSE_WAIT (remote closed, local app has not closed socket).

Production Application: CLOSE_WAIT accumulation indicates application resource leaks (open HTTP/database clients not calling close()), while excessive TIME_WAIT exhausts ephemeral ports under high connection churn.

- **DNS Resolution Hierarchy & Query Mechanics:** Step-by-step resolution from client stub resolver (/etc/resolv.conf) to local caching resolver, root DNS servers, TLD name servers, and authoritative name servers.

Production Application: In Kubernetes, ndots:5 configuration issues cause high CoreDNS latency by appending search domains up to 5 times for external FQDNs unless trailing dots are used.

- **Connection Tracking (conntrack) & NAT:** Linux netfilter subsystem tracking state for every bidirectional network stream. Required for stateful firewalling and Kubernetes kube-proxy iptables service routing.

Production Application: When conntrack table fills up (nf_conntrack: table full, dropping packet), nodes drop all new TCP connections. Requires tuning nf_conntrack_max and hash size.

- **TLS/SSL Cryptographic Handshake & PKI:** Asymmetric key exchange (ECDHE) and certificate validation against trusted Certificate Authorities (CAs), followed by symmetric session encryption (AES-GCM/ChaCha20).

Production Application: Automating cert-manager rotation in production to avoid catastrophic outages caused by expired wildcard SSL/TLS certificates.

3. Exhaustive Command & Syntax Reference

Command & Syntax	Purpose & Options	Production Usage & Common Pitfall
ss -tulpn ss [options]	Display all active listening TCP and UDP sockets along with bound IP addresses, ports, and owning process PIDs. Options: [{ 'flag': '-t', 'description': 'Display TCP sockets.' }, { 'flag': '-u', 'description': 'Display UDP sockets.' }, { 'flag': '-l', 'description': 'Filter for listening sockets only.' }, { 'flag': '-p', 'description': 'Show process name and PID holding the socket.' }, { 'flag': '-n', 'description': 'Numeric port and IP output (skip DNS resolution).'}]	Execute during deployment verification or port conflict triage to find which process holds port 80/443/8080. Pitfall: Omitting -n on busy servers, causing ss to freeze trying to reverse-resolve thousands of remote IP addresses.
curl -lv --resolve internal.svc:443:10.0.1.5 https://internal.svc/health curl [options]	Perform a detailed TLS handshake and HTTP header inspection overriding DNS resolution directly to a specific target IP. Options: [{ 'flag': '-I', 'description': 'Fetch HTTP response headers only (HEAD request).'}, { 'flag': '-v', 'description': 'Verbose mode showing TLS handshake, cert chain, and request headers.' }, { 'flag': '--resolve', 'description': 'Force DNS resolution mapping for hostname:port:ip.'}]	Execute to verify new SSL certificates or ingress routes without modifying public DNS records. Pitfall: Testing with -k (insecure) in production validation, masking invalid certificate chains or hostname mismatches.
tcpdump -i eth0 -nn -s 0 -w /tmp/packet_dump.pcap 'port 443 and host 10.0.0.15' tcpdump -i [options] "	Capture full-payload network packets on interface eth0 matching specific port and IP filter, saving to a Wireshark-compatible pcap file. Options: [{ 'flag': '-i eth0', 'description': 'Capture on network interface eth0.' }, { 'flag': '-nn', 'description': 'Do not resolve hostnames or port numbers to names.' }, { 'flag': '-s 0', 'description': 'Capture entire packet length (snaplen 0).'}, { 'flag': '-w', 'description': 'Write raw packets to output file.'}]	Run during active network incidents to capture raw packet traces for deep protocol analysis in Wireshark. Pitfall: Capturing without a targeted BPF filter on a high-throughput 10Gbps link, which exhausts disk space in seconds and drops packets.

Command & Syntax	Purpose & Options	Production Usage & Common Pitfall
sudo ufw default deny incoming && sudo ufw allow in on eth0 to any port 22 proto tcp <code>ufw [default allow deny] [rules]</code>	Establish a secure default-deny firewall posture while explicitly permitting SSH on the primary interface. Options: [{ 'flag': 'default deny incoming', 'description': 'Block all unsolicited inbound traffic.' }, { 'flag': 'allow in on eth0', 'description': 'Scope permit rule strictly to primary interface.' }, { 'flag': 'proto tcp', 'description': 'Restrict rule to TCP protocol.' }]	Execute as the first step during cloud VM provisioning to enforce zero-trust perimeter defense. Pitfall: Enabling UFW before allowing SSH port 22, permanently locking the administrator out of the remote instance.
ssh-keygen -t ed25519 -C 'devops_prod_deployer' -f ~/.ssh/id_ed25519 <code>ssh-keygen -t -C " " -f</code>	Generate modern, high-security Ed25519 elliptic-curve SSH keypair with descriptive comment metadata. Options: [{ 'flag': '-t ed25519', 'description': 'Use modern Ed25519 elliptic curve signature algorithm.' }, { 'flag': '-C', 'description': 'Add informative comment identifying key owner and environment.' }, { 'flag': '-f', 'description': 'Output path for the private key.' }]	Generate dedicated key pairs for each environment (staging, production) rather than sharing a single global key. Pitfall: Using obsolete RSA 1024/2048 keys or leaving private keys without passphrase protection in shared environments.

4. Security Hardening & Anti-Patterns

Security Mandates: Always operate under least-privilege principles. Restrict credentials, mask secrets in build outputs, avoid wildcard IAM permissions, and validate inputs before running commands.

Critical Anti-Pattern: Avoid manual drift ('click-ops' in cloud consoles or unversioned hotfixes in staging/production). Every change must be declared in code and reviewed via CI/CD.